Grafi

Materia	Algoritmi
Anno	Secondo Anno
Data	@December 13, 2024
Studiato	Done

Definiamo un grafo come la coppia $G = (V, E)$ dove V è l'insieme di vertici ed E è un insieme di coppie di vertici:

- se tali coppie sono non ordinate, ossia $(i, j) = (j, i)$, si ha un grafo **non orientato**;
- se tali coppie sono ordinate, ossia $(i, j) \neq (j, i)$, si ha un grafo **orientato**. Nel caso di un grafo orientato, specifichiamo per ogni arco la direzione tramite una freccia.

Poiché il grafo non è altro che una struttura dati che rappresenta relazioni, $V = \{ v_1, ..., v_n \}$ rappresenta degli elementi e E rappresenta la relazione tra vari elementi. Quindi in base alla relazione, possiamo decidere se il grafo è orientato o meno. Ad esempio, se la relazione è "i è fratello di j" conviene usare un grafo non direzionato, in quanto se i è fratello di j allora j è fratello di i. Invece, se la relazione è "i è più grande di j", chiaramente non vale il contrario, quindi conviene usare un grafo direzionato.

Un grafo si dice denso se $|E| = \Theta(|V|^2)$, sparso se $|E| = O(|V|)$.

Rappresentazione di un grafo

Possiamo rappresentare un grafo in due modi:

- tramite **matrici di adiacenza**;
- tramite **liste di adiacenza**.

Nella prima rappresentazione, si ha che $M[i, j] = 1$ se esiste un arco da i a j, 0 altrimenti. Nella seconda rappresentazione, si ha che ad ogni vertice corrisponde

una lista, in cui la presenza di un elemento nella lista indica che il vertice è adiacente a quest'ultimo.

Algoritmo di ricerca in ampiezza (Breadth First Search)

Un cammino non è altro che una sequenza di archi o di vertici adiacenti. In particolare, si ha che la sequenza $< u_1, ..., u_k >$ è un cammino se:

- $u_i \in V \ \ \forall i \in \{ 1, ..., k \}$;
- $(u_i, u_{i+1}) \in E \ \ \forall i \in \{ 1, ..., k-1 \}$.

Un cammino si dice ciclo semplice se $u_1 = u_k$.
Parleremo principalmente di
grafi etichettati.

> *Un grafo si dice etichettato se esiste una funzione:*
>
> $$w : E \to \mathbb{R}$$

Molto spesso w è un valore positivo, ma ci sono anche dei casi particolari in cui w è negativo.

Per la ricerca **BFS**, ossia la **ricerca in ampiezza**, non ci interessa l'etichetta w. Possiamo considerare ogni arco con stesso peso $w = 1$. Partiamo da un **nodo sorgente** generico: non ha importanza da dove partiamo. Una conseguenza della visita BFS naturale è il **calcolo delle distanze di cammino minimo**. L'algoritmo BFS serve per visitare nodi ma può anche trovare dei cammini minimi. Di conseguenza può essere usato per trovare i cammini minimi dal nodo sorgente.
Il primo passo è quello di visitare tutti i nodi a distanza
1 dal nodo sorgente. Poi si visitano i nodi a distanza 2 e così via. Si procede quindi per livelli, analizzando prima i nodi vicini e poi quelli lontani.

Si utilizza per tale algoritmo una coda Q, dove man mano verranno inseriti gli elementi rispetto alla loro disposizione topologica. Possiamo etichettare un nodo in modo tale che abbiamo informazione per quando il nodo viene visitato e per quando la visita viene completata. Introduciamo quindi un campo **colore**, che può essere:

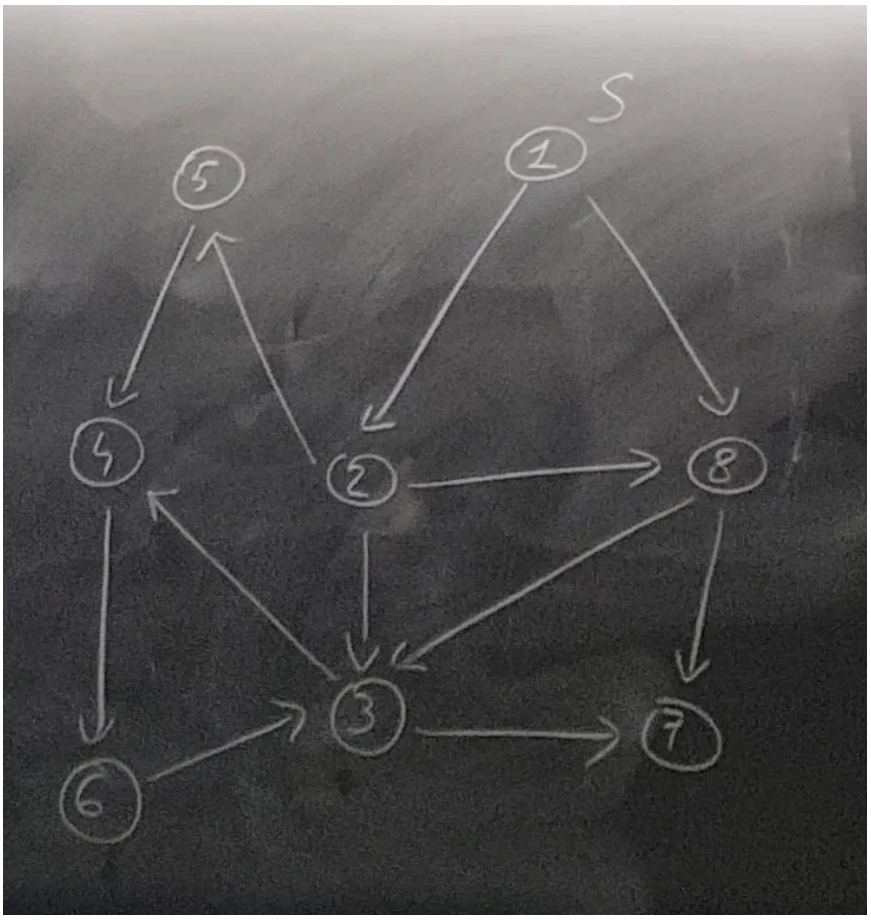
- **bianco**, se il nodo non è stato visitato, di cui quindi non conosciamo ancora l'esistenza;
- **grigio**, se il nodo ha iniziato la visita ma non è terminata;
- **nero**, se la visita è stata completata, quindi non verranno visti successivamente.

Aggiungiamo anche una nuova etichetta $\pi[v]$ che rappresenta il **predecessore di v nell'albero BFS che si genera con l'esplorazione del grafo a partire dal nodo sorgente**. L'albero BFS permette di comprendere la distanza di cammino minimo dal nodo sorgente ai nodi raggiungibili. Nell'albero BFS si possono avere archi da un ramo ad un altro, ma non archi che vanno da un nodo ad un suo discendente non diretto, altrimenti sarebbe in un livello precedente.

Gli alberi BFS inoltre non sono unici, ma ci possono essere anche più alberi BFS che indicano la stessa esplorazione: l'unica cosa che cambia è la scelta del primo nodo esplorato in un livello k. I collegamenti alternativi non sono importanti se sono sullo stesso livello.

$\pi[v]$ permette di avere informazioni sull'albero, in particolare ci dice chi è il padre del nodo v.

Il modo in cui si usa la coda Q è il seguente. Si parte dal nodo sorgente, e per ogni nodo si inseriscono i figli in coda. Quando vengono inseriti in coda, il colore del nodo diventa grigio. Diventeranno neri quando vengono rimossi da tale coda. Ad esempio, nel grafo sottostante:



Partendo da uno, inseriamo in coda il nodo 1. Poi inseriamo in coda i suoi adiacenti, ossia 2 e 8. Una volta averli inseriti, rimuoviamo il nodo 1: esso diventerà nero. Poi passiamo al 2, e inseriamo i suoi adiacenti, ossia 3 e 5. Il 2 viene rimosso e diventa nero. Poi si passa al nodo 8, e inseriamo nuovamente il 3 e poi il 7. Il nodo 8 diventa nero. Si procede così allo stesso modo per ogni nodo.

Scriviamo allora la procedura:

```
BFS(V, s):
   for each v in V do
      color[v] = white
      pi[v] = null
      d[v] = inf
   d[s] = 0
   color[s] = gray
   Q = {s}
```

```
while Q is not empty do
   u = dequeue(q)
   for each v in adj[u] do
      if color[v] = white then
        color[v] = gray
        d[v] = d[u] + 1
        enqueue(Q, v)
        pi[v] = u
   color[u] = black
```

L'array d memorizza la distanza di ogni nodo da s. Solitamente la distanza viene inizializzata ad un valore valido per tutti. In alcuni casi alcuni nodi sono irraggiungibili. Pertanto la distanza è inizializzabile con valore infinito, proprio per questi casi.

$adj[u]$ rappresenta la **lista di adiacenza di** u. Se il grafo invece è rappresentato tramite matrice di adiacenza, il ciclo for viene rappresentato in maniera diversa. In una lista di adiacenza la complessità può essere più piccola rispetto alla matrice di adiacenza.

Analizziamo la complessità, considerando $|V| = n, |E| = m$:

- il ciclo for di inizializzazione richiede un tempo $O(n)$, indipendente dall'implementazione del grafo;
- il ciclo while della coda Q richiede al massimo n iterazioni, in quanto **ogni nodo entra in coda solo** 1 **volta** per il semplice fatto che **solo i nodi bianchi entrano in coda**. Il ciclo quindi ha una complessità $O(n)$;
- il ciclo for each all'interno del while nel caso pessimo richiede $O(n)$: questo perché la lista di adiacenza potrebbe contenere tutti i nodi del grafo. Ci sono tuttavia casi migliori proprio per la lista di adiacenza. La somma delle esecuzioni del ciclo for each infatti è il **numero di archi**: in una lista di adiacenza infatti ci saranno tanti nodi quanti sono gli archi uscenti. Di conseguenza, complessivamente l'esecuzione richiede $O(m)$. Il ciclo while in totale richiede $O(n + m)$, con $m \leq n^2$.

Queste osservazioni tuttavia sono valide se usiamo una lista di adiacenza. Con una matrice di adiacenza, in realtà, la complessità si alza a $O(n^2)$.

Mentre attraversiamo il grafo, dobbiamo costruire l'albero dei predecessori. Allora possiamo salvare il predecessore del vertice v nel ciclo for each e impostare $\pi[v] = u$.

Riprendendo il grafo precedente, eseguiamo l'algoritmo. All'inizio gli array d, π sono inizializzati a ∞ e null. Il colore è bianco per tutti. Consideriamo $s = 1$. Il suo colore diventa grigio, distanza 0 e $\pi[1] = \text{null}$. Inseriamo in coda $2, 8$ e li marchiamo di grigio. Abbiamo finito con 1, quindi coloriamo di nero.

Passiamo al nodo 2, che ha $d = 1, \pi = 1$ e inseriamo in coda il 5 e il numero 3, che hanno $d = 2$ e $\pi = 2$. Tali nodi saranno colorati in grigio e il 2 verrà colorato di nero. Poi passiamo al nodo 8. Il nodo 8 ha $d = 1, \pi = 1$ e inseriamo in coda il nodo 7 e lo coloriamo di grigio, mentre 8 diventa nero. Estraiamo il nodo 3 dalla coda, che inserisce in coda il nodo 4: il 4 viene colorato di grigio ed è tale che $d = 3, \pi = 3$. Il nodo 3 diventa nero. Passiamo al 5, che non inserisce in coda nulla in quanto il 4 era già grigio. Allora viene colorato di nero. Consideriamo il 7, che viene colorato subito di nero in quanto non abbiamo nessun nodo che possiamo esplorare. Anche per il 7, $d = 3, \pi = 3$. Infine, il 4 inserisce in coda il 6, che ha $\pi = 4, d = 4$. I nodi restanti verranno poi colorati in nero.

Possiamo calcolare la distanza di un nodo dalla radice dell'albero con la seguente procedura:

```
distance(pi, v)
   if pi[v] == null then return 0
   return 1 + distance(pi, pi[v])
```

Ricerca in profondità (Depth First Search)

La ricerca in profondità esplora prima un ramo e poi fa retromarcia.
La DFS può essere usata per il calcolo dell'
ordinamento topologico di un grafo.

Supponiamo di avere un grafo orientato in cui ogni nodo rappresenta una materia. La relazione tra queste attività identificata dagli archi è la propedeuticità. Dobbiamo trovare una sequenza di passi che permette di trovare una laurea. Tale grafo dev'essere **aciclico**.

L’ordinamento topologico è una disposizione lineare dei nodi in maniera tale che valga la proprietà $(i, j) \in E : i < j$.